FacePass API

Integration guide for web and mobile applications

API version v1 (service 0.1.0) · November 2026

This guide is for developers connecting a website or mobile app to FacePass. It explains how to enroll people, check a photo against them, and receive results securely. You can call the API from any language that can send HTTPS requests.

Contents

Before you start . . . 1

Choose an integration flow . . . 2

Create a subject first . . . 2

Client upload flow . . . 2

Backend upload flow . . . 5

Webhooks . . . 6

Choosing verify or identify . . . 7

API reference . . . 7

Error handling . . . 8

Before you go live . . . 9

Before you start

Get these from whoever runs your FacePass service:

- **Base URL:** this guide uses `https://facepass.example.com`. Replace it with your own.
- **API key:** it starts with `fp_key_`, and it gives full access to your application's data.
- **Webhook secret (optional):** needed only if you want results pushed to your backend.
- **Allowed origins (web only):** ask them to add your site's origin to the CORS settings.

> **Never put an API key in a website or app.** Anyone can extract it and then read or delete your people's data. Keep it on your backend, and give clients short-lived session tokens instead.

Every request except the health checks sends a credential as `Authorization: Bearer <value>`. FacePass only needs your own user id for each person, so avoid sending names or email addresses.

Core concepts

- **Subject:** a person in your application. Your own user id goes in `external_id`.
- **Face:** a stored numeric template of a person's face. Photos are discarded after processing.
- **Verify (1:1):** does this photo match this specific person?
- **Identify (1:N):** who among all the enrolled people does this photo show?
- **Session:** a short-lived permission that lets a client upload one photo directly.

Choose an integration flow

	Client upload (session)	Backend upload
Path of the photo	Client → FacePass	Client → your backend → FacePass
Credential	Short-lived session token on the client	API key on your backend
Result goes to	Your backend only	Your backend
Best when	You don't want photos passing through your servers	You already handle uploads, or the check runs server-side

Most web and mobile apps should use the **client upload** flow. Both flows use the same subjects and faces.

Create a subject first

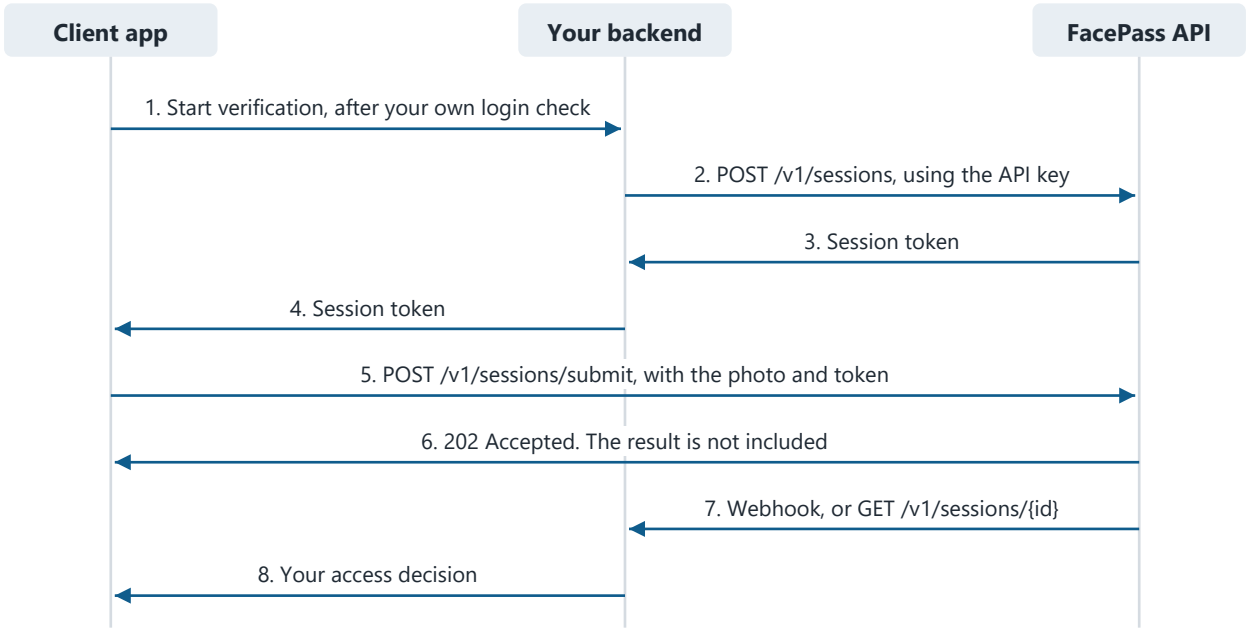
Both flows start here. Call this from your backend once per person, and store the returned `id` with your user record.

```
curl https://facepass.example.com/v1/subjects \
  -H "Authorization: Bearer $FACEPASS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"external_id": "user-123"}'
```

```
{
  "id": "b7c4…",
  "external_id": "user-123",
  "created_at": "2026-11-01T12:00:00Z"
}
```

To look someone up later, call `GET /v1/subjects?external_id=user-123`. An `external_id` that already exists returns `409 external_id_conflict`.

Client upload flow



The client never receives the match result, so a modified app can't fake one. **Make access decisions only from what your backend reads from FacePass.**

1. Create a session on your backend

Authenticate your own user first, and only then create a session for that user's subject.

```
curl https://facepass.example.com/v1/sessions \
  -H "Authorization: Bearer $FACEPASS_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"action": "verify", "subject_id": "b7c4…"}'
```

```
{
  "id": "3f1a…",
  "token": "fp_ses_…",
  "action": "verify",
  "subject_id": "b7c4…",
  "expires_at": "2026-11-01T12:02:00Z",
  "max_attempts": 3
}
```

Return only the token to the client. The request body depends on the action:

Action	Request body	Use it to
`enroll`	`{"action": "enroll", "subject_id": "…", "consent": true}`	Add a face. Send consent only after the person has agreed
`verify`	`{"action": "verify", "subject_id": "…"}`	Check one person
`identify`	`{"action": "identify"}`	Search all enrolled people

2. Upload the photo from the client

Send one multipart form field named `image`, and use the session token as the Bearer credential.

Web (JavaScript)

```
const stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode: "user" } });
// ...attach the stream to a <video> element, then capture a frame when it's playing:
const canvas = document.createElement("canvas");
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;
canvas.getContext("2d").drawImage(video, 0, 0);
const blob = await new Promise((resolve) => canvas.toBlob(resolve, "image/jpeg", 0.92));

const form = new FormData();
form.append("image", blob, "face.jpg");
const response = await fetch(`${FACEPASS_URL}/v1/sessions/submit`, {
  method: "POST",
  headers: { Authorization: `Bearer ${token}` },
  body: form,
});
```

Browsers allow camera access only over HTTPS, except on `localhost`. Your site's origin must also be on the service's allowed list.

> **Mobile samples are untested.** The Android, iOS and Flutter samples below show the correct request shape, but they haven't been run in real apps yet. The web flow has been tested in Chrome. Test on real devices before release.

Android (Kotlin with OkHttp)

```
val body = MultipartBody.Builder().setType(MultipartBody.FORM)
    .addFormDataPart("image", "face.jpg", jpegBytes.toRequestBody("image/jpeg".toMediaType()))
    .build()
val request = Request.Builder()
    .url("$facepassUrl/v1/sessions/submit")
    .header("Authorization", "Bearer $token")
    .post(body)
    .build()
```

iOS (Swift)

```
var request = URLRequest(url: facepassURL.appendingPathComponent("v1/sessions/submit"))
request.httpMethod = "POST"
request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
let boundary = UUID().uuidString
request.setValue("multipart/form-data; boundary=\(boundary)", forHTTPHeaderField: "Content-Type")
var body = Data()
body.append(Data("--\(boundary)\r\n".utf8))
body.append(Data("Content-Disposition: form-data; name=\"image\"; filename=\"face.jpg\"\r\n".utf8))
body.append(Data("Content-Type: image/jpeg\r\n\r\n".utf8))
body.append(jpegData)
body.append(Data("\r\n--\(boundary)--\r\n".utf8))
let (data, response) = try await URLSession.shared.upload(for: request, from: body)
```

Flutter (http package)

```
final request = http.MultipartRequest('POST', Uri.parse('$facepassUrl/v1/sessions/submit'))
  ..headers['Authorization'] = 'Bearer $token'
  ..files.add(http.MultipartFile.fromBytes('image', jpegBytes, filename: 'face.jpg'));
final response = await request.send();
```

Handle the upload response

Status	Meaning	What the client should do
`202`	The upload was accepted	Ask your backend for the decision
`422`	The photo was rejected, such as for no face or failed liveness	Show `detail` and let the user retake it. The session allows `max_attempts` uploads
`409`	The session is already closed	Ask your backend for a new session
`410`	The session expired (after 2 minutes by default)	Ask your backend for a new session

Camera tips

- Use the front camera, and capture JPEG, PNG or WebP. iOS HEIC photos aren't accepted, so convert them to JPEG first.
- Keep the whole face in frame and at least 64 pixels wide. Aim for 640 pixels or more on the image's long edge.
- Keep uploads under 8 MB. FacePass applies EXIF rotation and downscales large photos for you.

3. Get the result on your backend

Either call `GET /v1/sessions/{id}` with your API key, or receive a webhook. A session's `status` is one of `pending`, `completed`, `failed` or `expired`.

```
{
  "id": "3f1a…",
  "action": "verify",
  "status": "completed",
  "subject_id": "b7c4…",
  "attempts": 1,
  "result": {
    "match": true,
    "subject_id": "b7c4…",
    "score": 0.6714,
    "threshold": 0.4,
    "model_id": "sface-2021dec",
    "liveness": { "checked": true, "score": 0.97 }
  },
  "created_at": "2026-11-01T12:00:00Z",
  "expires_at": "2026-11-01T12:02:00Z",
  "finished_at": "2026-11-01T12:00:40Z"
}
```

Check `status` first, then `result.match`. A `failed` session holds the last rejection, such as `{"error": "no_face_detected"}`. An enroll session's result holds `enrolled`, `face_id` and `subject_id`.

Backend upload flow

If your backend already has the photo, call these endpoints directly with your API key. The results come back in the response. In identify results, `candidates` lists people at or above the threshold, best first, and is empty when nobody matches.

Enroll a face

```
curl https://facepass.example.com/v1/subjects/b7c4…/faces \
  -H "Authorization: Bearer $FACEPASS_API_KEY" \
  -F image=@photo.jpg \
  -F consent=true
```

Verify (1:1)

```
curl https://facepass.example.com/v1/verify \
  -H "Authorization: Bearer $FACEPASS_API_KEY" \
  -F image=@photo.jpg \
  -F subject_id=b7c4…
```

Identify (1:N)

```
curl https://facepass.example.com/v1/identify \
  -H "Authorization: Bearer $FACEPASS_API_KEY" \
  -F image=@photo.jpg
```

Identify response

```
{
  "match": true,
  "candidates": [
    { "subject_id": "b7c4…", "external_id": "user-123", "score": 0.6714 }
  ],
  "threshold": 0.45,
  "model_id": "sface-2021dec",
  "liveness": { "checked": true, "score": 0.97 }
}
```

Webhooks

The service operator sets your webhook URL, and this also creates the signing secret. Changing it replaces the old secret:

```
facepass apps set-webhook <application-id> https://your-backend.example.com/facepass/webhook
```

You'll receive `session.completed`, `session.failed` and `session.expired` events:

```
{
  "id": "6e0d…",
  "type": "session.completed",
  "created_at": "2026-11-01T12:00:40+00:00",
  "data": {
    "session_id": "3f1a…",
    "action": "verify",
    "status": "completed",
    "subject_id": "b7c4…",
    "result": { "match": true, "score": 0.6714, "threshold": 0.4 }
  }
}
```

The `result` above is shortened. It has the same fields as the session's result.

Verify the signature

Check every webhook before trusting it. The `FacePass-Signature` header covers the raw request body, so check it before parsing the JSON:

```
import { createHmac, timingSafeEqual } from "node:crypto";

function verifyFacePassSignature(rawBody, header, secret) {
  const fields = Object.fromEntries(header.split(",").map((part) => part.split("=")));
  const expected = createHmac("sha256", secret).update(`${fields.t}.${rawBody}`).digest("hex");
  const fresh = Math.abs(Date.now() / 1000 - Number(fields.t)) < 300;
  const a = Buffer.from(expected);
  const b = Buffer.from(fields.v1 ?? "");
  return fresh && a.length === b.length && timingSafeEqual(a, b);
}
```

- **Respond quickly:** return a 2xx within 10 seconds, and do slow work afterward.
- **Expect retries:** failures retry with exponential backoff, up to 8 attempts. Delivery is at-least-once, so ignore repeats of a `FacePass-Event-Id` you've already processed.
- **Don't redirect:** redirects count as failures, because signed payloads go only to the configured URL.

Choosing verify or identify

Prefer **verify** whenever you know who the person claims to be, such as after a username or from a QR code. It's more accurate, and false matches don't grow with the number of enrolled people.

Identify compares against everyone in your application, so its false match risk rises as you enroll more people. It uses a stricter threshold to compensate. Test it with your real number of enrolled people before relying on it.

Enrollment tips

- Enroll 2–3 photos per person, in different lighting, to improve matching.
- A subject can have up to 5 faces by default. Each new face must match the ones already enrolled, or you'll get `409 face_mismatch`.

API reference

Endpoint	Credential	Purpose
`POST /v1/subjects`	API key	Create a person
`GET /v1/subjects`	API key	List people, or find one with `external_id`. Uses `limit` and `cursor` for paging
`GET /v1/subjects/{id}`	API key	Read a person
`DELETE /v1/subjects/{id}`	API key	Permanently delete a person and their face data
`POST /v1/subjects/{id}/faces`	API key	Enroll a face
`GET /v1/subjects/{id}/faces`	API key	List enrolled faces
`DELETE /v1/subjects/{id}/faces/{face_id}`	API key	Delete one face
`POST /v1/verify`	API key	Check a photo against one person
`POST /v1/identify`	API key	Find who a photo shows
`POST /v1/sessions`	API key	Create a client upload session
`GET /v1/sessions/{id}`	API key	Read a session's outcome
`POST /v1/sessions/submit`	Session token	Upload a photo from a client
`GET /healthz`	None	Check that the service is running
`GET /readyz`	None	Check that the service can handle traffic

Interactive documentation is at `/docs` on your FacePass service, and the OpenAPI specification at `/openapi.json` can generate client libraries.

Default limits

Limit	Default
Upload size	8 MB, and 40 megapixels
Image formats	JPEG, PNG and WebP
Session lifetime	2 minutes
Uploads per session	3. A conclusive result closes the session earlier
Requests per API key	120 per minute. See the `RateLimit-*` response headers
Failed verifications per subject	5, then verification locks for up to 15 minutes
Faces per subject	5
Verify threshold / identify threshold	0.40 / 0.45 (cosine similarity)

The service operator can change these, so check with them if yours may differ.

Error handling

Errors use `application/problem+json` (RFC 9457). Branch on `code`, and quote `request_id` when reporting a problem.

```
{
  "type": "urn:facepass:problem:no_face_detected",
  "title": "No face detected",
  "status": 422,
  "code": "no_face_detected",
  "detail": "Center one face in the frame, facing the camera.",
  "request_id": "9f2c…"
}
```

Code	Status	What to do
`no_face_detected`, `multiple_faces_detected`, `face_out_of_frame`, `face_too_small`, `face_too_blurry`, `poor_lighting`	422	Show `detail` and let the user retake the photo
`liveness_failed`	422	Ask for a live photo. Repeated failures during verify lock the subject
`consent_required`	422	Collect consent, then send it as `true`
`validation_error`	422	Fix the request. `errors` lists the fields
`unauthorized`	401	Check the credential and that it's the right type for this endpoint
`subject_not_found`, `face_not_found`, `session_not_found`	404	Check the id. Other applications' data also appears as not found
`face_mismatch`	409	The face doesn't match this subject's enrolled faces
`face_limit_reached`	409	Delete a face before adding another
`subject_not_enrolled`	409	Enroll a face first
`external_id_conflict`	409	Use the existing subject
`session_closed`	409	Create a new session

Code	Status	What to do
`session_expired`	410	Create a new session
`length_required`	411	Send a Content-Length header. Most HTTP libraries do this for you
`image_too_large`	413	Send a smaller image
`unsupported_image`	415	Send a valid JPEG, PNG or WebP
`empty_image`	400	Send a non-empty file
`invalid_cursor`	400	Pass the `next_cursor` value unchanged
`too_many_failed_attempts`	429	Verification is locked. Wait for `Retry-After`
`rate_limited`	429	Slow down. Wait for `Retry-After`
`service_busy`	503	Retry after `Retry-After`

Before you go live

> **The liveness check isn't validated against real attacks.** Scanned photographs passed it at the default threshold during evaluation, and a client can upload any file. Don't use face matching as the only factor for payments, account recovery or anything high-stakes. Add a second factor, such as a PIN or trusted device.

Checklist

- **HTTPS:** use HTTPS everywhere, since plain HTTP must never carry face data across a network.
- **Secrets:** keep API keys and webhook secrets on your backend, and never log them.
- **Decisions:** act only on results your backend reads from FacePass, never on what a client reports.
- **Accuracy:** test on your real devices and users. Benchmark accuracy is 99.40% on the LFW press photo dataset, and your results will differ.
- **Fallback:** offer a non-biometric alternative for people who can't or won't use their face.

Privacy and consent

Face data is regulated as biometric data in many places, including under the GDPR and Illinois BIPA, so get legal advice for your launch regions. This isn't legal advice, but you'll typically need to:

- Tell people what you collect, why, how long you keep it and who processes it.
- Get explicit consent before enrolling. Sending consent as true is your statement that this happened.
- Call `DELETE /v1/subjects/{id}` when someone withdraws consent or closes their account.
- Delete the data of inactive users on a schedule, because stored faces don't expire automatically.